

✓ 04_lstm.ipynb

LSTM Model for Web Traffic Forecasting (60-Day Horizon)

This notebook loads the engineered feature dataset from Notebook 02 and trains a simple LSTM model to forecast the next **60 days** for each page.

Steps:

- Load features and labels
- Create sequences suitable for LSTM
- Build an LSTM model (TensorFlow / Keras)
- Train for a small number of epochs
- Generate predictions
- Compute SMAPE metric
- Save outputs for ensemble (Notebook 05)

✓ Imports

```
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from sklearn.preprocessing import MinMaxScaler
import matplotlib.pyplot as plt
import os
```

✓ Mount Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

✓ Load Features + Labels

We load:

- features.csv → lag + rolling + metadata
- labels.csv → next 60 days (our prediction targets)

```
save_dir = "/content/drive/MyDrive/CSC480/Final Project/"
```

```
features_path = save_dir + "features.csv"
```

```
labels_path = save_dir + "labels.csv"
```

```
df_features = pd.read_csv(features_path)
```

```
df_labels = pd.read_csv(labels_path)
```

```
df_features.head(), df_labels.head()
```

```
2015-07-03 2015-07-04 2015-07-05 2015-07-06 2015-07-07 2015-07-08 \
0 1.609438 1.945910 2.197225 1.945910 1.609438 0.000000
1 5.736572 5.429346 5.774552 5.743003 5.493061 5.468060
2 2.995732 3.433987 3.091042 3.218876 2.890372 5.187386
3 6.652863 7.060476 6.304449 6.628041 6.320768 6.204558
4 7.441320 7.358831 7.336286 7.363914 7.383368 7.457032

2015-07-09 ... 2016-12-25_roll_std 2016-12-26_roll_std \
0 1.098612 ... 0.506236 0.514896
1 5.497168 ... 0.169041 0.190357
2 3.713572 ... 0.259850 0.429010
3 8.476788 ... 0.196998 0.198785
4 7.560080 ... 0.046309 0.089099

2016-12-27_roll_std 2016-12-28_roll_std 2016-12-29_roll_std \
0 0.534076 0.270985 0.270985
1 0.196959 0.220090 0.214486
2 0.440246 0.421489 0.425851
3 0.233202 0.232868 0.233754
4 0.095800 0.098498 0.098561

2016-12-30_roll_std 2016-12-31_roll_std lang_enc access_enc agent_enc
0 0.270985 0.216804 248 2 0
1 0.230481 0.233897 236 0 1
2 0.494383 0.547694 337 2 0
3 0.232204 0.269482 341 0 1
4 0.097406 0.102299 223 2 1

[5 rows x 3307 columns],
2016-11-02 2016-11-03 2016-11-04 2016-11-05 2016-11-06 2016-11-07 \
0 2.944439 1.945910 1.945910 1.609438 1.386294 2.484907
1 6.115892 6.061457 5.726848 5.852202 6.061457 5.910797
2 1.386294 1.945910 1.098612 1.609438 0.000000 2.564949
3 6.706862 6.558198 6.844815 6.315358 6.648985 6.626718
4 8.643650 8.463370 8.562167 8.631236 8.684909 8.499029

2016-11-08 2016-11-09 2016-11-10 2016-11-11 ... 2016-12-22 \
0 2.197225 1.945910 1.791759 2.197225 ... 1.945910
1 5.823046 5.676754 5.886104 5.652489 ... 5.446737
2 1.791759 0.693147 1.098612 0.693147 ... 1.945910
```

3	5.686975	5.883322	5.323010	5.780744	6.084499	5.846439
4	7.723120	7.709757	7.635787	7.928406	7.845024	7.833996

	2016-12-29	2016-12-30	2016-12-31
0	1.945910	1.945910	2.302585
1	5.594711	5.308268	5.252273
2	1.609438	2.484907	2.564949
3	5.703782	5.726848	5.356586
4	7.765993	7.737180	7.675082

[5 rows x 60 columns])

✓ LSTM Input Format

LSTM expects data in the format:

(samples, timesteps, features)

We will reshape the engineered features into:

- timesteps = 1
- features = all numeric columns

This gives the LSTM the ability to process all input features per page.

```
# Select numeric feature columns
X = df_features.select_dtypes(include=[np.number]).values
y = df_labels.values # shape: (samples, 60)

print("Original X shape:", X.shape)
print("Original y shape:", y.shape)

# LSTM expects 3D: (samples, timesteps=1, features)
X_lstm = np.expand_dims(X, axis=1)

print("Reshaped X for LSTM:", X_lstm.shape)
```

```
Original X shape: (500, 3303)
Original y shape: (500, 60)
Reshaped X for LSTM: (500, 1, 3303)
```

✓ LSTM Model Architecture

We use a simple architecture:

- LSTM(64)
- Dropout(0.2) to avoid overfitting
- Dense(60) output layer (one value per day in forecast)

```
model = Sequential([
    LSTM(64, input_shape=(1, X.shape[1])),
```

```

        Dropout(0.2),
        Dense(60) # output horizon
    ])

model.compile(optimizer='adam', loss='mae')
model.summary()

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning
 super().__init__(**kwargs)

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 64)	862,208
dropout (Dropout)	(None, 64)	0
dense (Dense)	(None, 60)	3,900

Total params: 866,108 (3.30 MB)
Trainable params: 866,108 (3.30 MB)
Non-trainable params: 0 (0.00 B)

✓ Train the LSTM Model

We train for a small number of epochs (e.g., 5) due to limited time/resources.

This is enough for a baseline forecast used in ensemble.

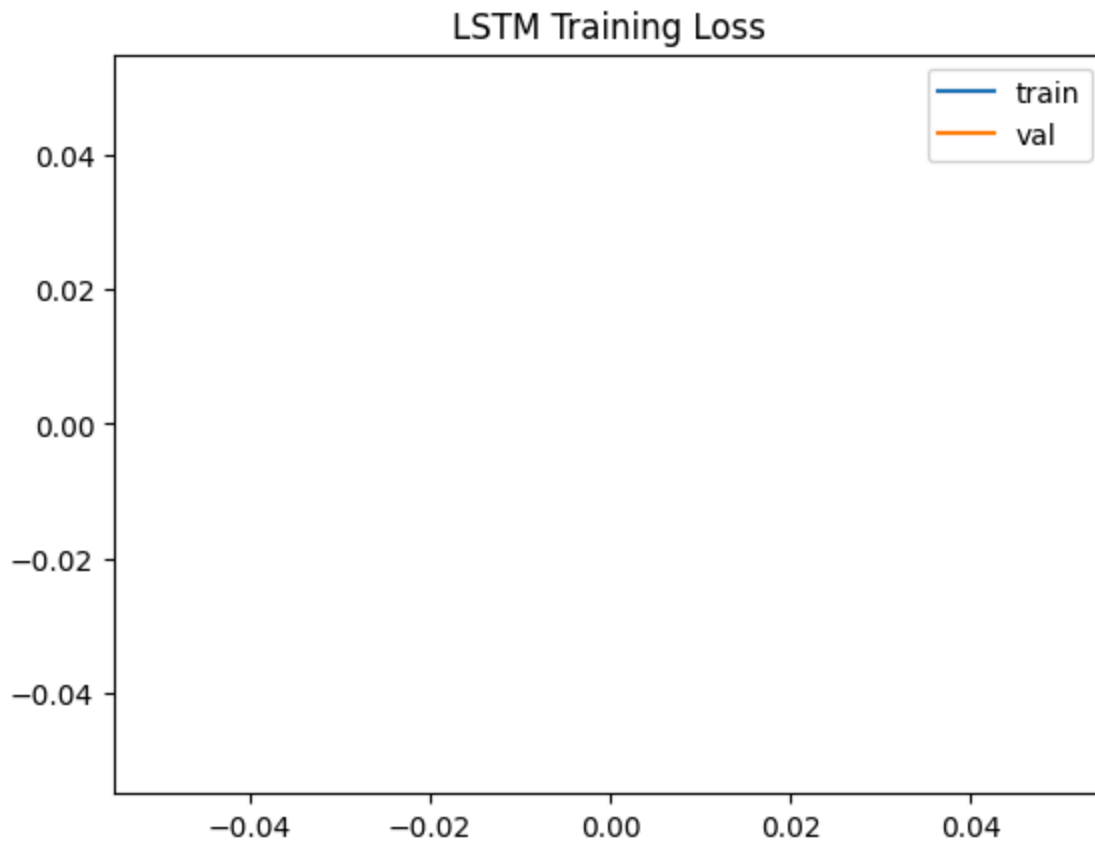
```

history = model.fit(
    X_lstm, y,
    epochs=5,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)

# Plot training curve
plt.plot(history.history["loss"], label="train")
plt.plot(history.history["val_loss"], label="val")
plt.title("LSTM Training Loss")
plt.legend()
plt.show()

```

Epoch 1/5
15/15 ————— **1s** 21ms/step - loss: nan - val_loss: nan
Epoch 2/5
15/15 ————— **0s** 9ms/step - loss: nan - val_loss: nan
Epoch 3/5
15/15 ————— **0s** 9ms/step - loss: nan - val_loss: nan
Epoch 4/5
15/15 ————— **0s** 9ms/step - loss: nan - val_loss: nan
Epoch 5/5
15/15 ————— **0s** 9ms/step - loss: nan - val_loss: nan



✓ Predict the Next 60 Days

We run the model on all LSTM inputs.

```
lstm_predictions = model.predict(X_lstm)
lstm_predictions.shape
```

16/16 ————— **0s** 9ms/step
(500, 60)

✓ Compute SMAPE for LSTM

```
def smape(true, pred):
    numer = np.abs(true - pred)
    denom = np.abs(true) + np.abs(pred) + 1e-8
    return np.mean(2 * numer / denom)
```

```
lstm_smape = smape(y, lstm_predictions)
print("LSTM SMAPE:", lstm_smape)
```

LSTM SMAPE: nan

✓ Save LSTM Outputs for Ensemble (Notebook 05)

- lstm_predictions.csv
- lstm_smape.txt

```
save_dir = "/content/drive/MyDrive/CSC480/Final Project/"
os.makedirs(save_dir, exist_ok=True)

# Save predictions
pred_path = save_dir + "lstm_predictions.csv"
np.savetxt(pred_path, lstm_predictions, delimiter=",")

# Save smape score
score_path = save_dir + "lstm_smape.txt"
with open(score_path, "w") as f:
    f.write(str(lstm_smape))

print("Saved LSTM predictions to:", pred_path)
print("Saved LSTM SMAPE score to:", score_path)
```

```
Saved LSTM predictions to: /content/drive/MyDrive/CSC480/Final Project/lstm_predictions.csv
Saved LSTM SMAPE score to: /content/drive/MyDrive/CSC480/Final Project/lstm_smape.txt
```